Software Architecture Document (SAD)

Clario

A Multi-Agent Customer Support Triage and Response System

Date: 29/07/2026
Authors: WEERASEKARA R.V. (230694J), WETTASINGHE V.N. (230701G), WICKRAMARATNA S.I.P. (230703N)
Group ID: Group 23 (Project ID: P04)
Mentor: Dr. Aloka Fernando
Teaching Assistant: Dilanka

Table of Contents

1. Introduction

The introduction of this Software Architecture Document provides an overview of the entire document. It includes the purpose, scope, definitions, acronyms, abbreviations, references, and overview of the Clario platform's software architecture.

1.1 Purpose

This document provides a comprehensive architectural overview of the system, using a number of different architectural views to depict different aspects of the system. It is intended to capture and convey the significant architectural decisions which have been made on the system. This document is the single source of truth for all architectural decisions. All cross-team contracts (state shapes, API schemas, JSON payloads) are defined here and must be communicated as a team before implementation.

1.2 Scope

This Software Architecture Document applies to the **Clario** platform: an end-to-end automated customer support triage and response pipeline. The system is built on a **multi-agent collaboration** pattern, deployed as a **Hybrid Monolit**h (a Spring Boot API gateway + a Python FastAPI ML Sidecar, containerised together via Docker Compose).

This document covers:

- Inbound ticket ingestion (Next.js UI + Spring Boot gateway)
- PII redaction via **SurrogateShield**
- Semantic distillation and LLM-based multi-label classification
- LangGraph state-graph orchestration (routing, drafting, validation, reflection, escalation)
- Retrieval-Augmented Generation (RAG) using ChromaDB
- Evidence Graph Consistency (EGC) validation
- Human-in-the-loop escalation and optimistic-locking conflict resolution
- QLoRA fine-tuning pipeline (offline, Kaggle/Colab GPU)

1.3 Definitions, Acronyms, and Abbreviations

- **EGC**: Evidence Graph Consistency.
- **LLM**: Large Language Model.
- **PII**: Personally Identifiable Information.
- **QLoRA**: Quantized Low-Rank Adaptation (fine-tuning method).
- **RAG**: Retrieval-Augmented Generation.
- **SurrogateShield**: A custom PII anonymization module.
- **Hybrid Monolith**: An architecture combining a Spring Boot gateway and a Python ML Sidecar running locally.
- **LangGraph**-: Framework for building stateful, multi-actor applications with LLMs.

1.5 Overview

This document is organised into 12 sections following the standard 4+1 View Model (Use-Case, Logical, Process, Deployment, and Implementation Views) supplemented with a Data View and Quality sections. Detailed pipeline documentation, the technology stack, testing strategy, ADRs, and

the project timeline are provided in the Appendices so that the primary 12-section body remains compliant with the supervisor's template structure.

2. Architectural Representation

The software architecture for Clario is represented using a mix of UML and C4 model diagrams:

- **Use-Case View**: Represented by UML Use-Case diagrams to show actor interactions and realizations.
- **Logical View**: Represented by UML Class diagrams and Component Context diagrams (C1/C2/C3) to show structural decomposition.
- **Process View**: Represented by UML Activity and Sequence diagrams to show runtime behavior and synchronization.
- **Deployment View**: Represented by UML Deployment diagrams showing physical nodes and containers.
- **Implementation View**: Represented by Package and Component diagrams.
- **Data View:** Entity-Relationship Diagram (ERD), Persistent data model and table relationships

3. Architectural Goals and Constraints

Goals:

- **Automate routine support:** Use a LangGraph multi-agent pipeline to classify, route, draft, and validate responses without human intervention for standard tickets.
- **Prevent PII leakage:** Ensure raw customer PII never reaches any external LLM or training dataset after SurrogateShield processing.
- **Grounded responses:** Ensure all agent drafts are grounded in the ChromaDB knowledge base via RAG with relevance thresholding; never hallucinate.
- **Safe concurrent editing:** Prevent lost updates when multiple human agents review the same escalated ticket simultaneously.
- **Reproducible evaluation:** A locked `test.parquet` split is evaluated exactly once; all metrics are repeatable.

Constraints:

Academic GPU only: Fine-tuning must run on Kaggle/Colab free-tier GPUs. QLoRA (4-bit quantisation + LoRA adapters) is mandatory.

Zero-capital cloud: No paid infrastructure -Docker Compose on a local machine; ChromaDB in a persistent volume.

Race-condition safety: Optimistic locking (`version` column + HTTP 409) is enforced at the database layer for all human-review `PATCH` operations.

No PII in training data: The offline `pii_clean.py` script masks all PII from the raw Kaggle dataset before distillation or fine-tuning begins.

4. Use-Case View

This section lists the architecturally significant use cases of Clario. These use cases have high architectural coverage -they exercise many architectural elements -and they stress the key design points of the system.

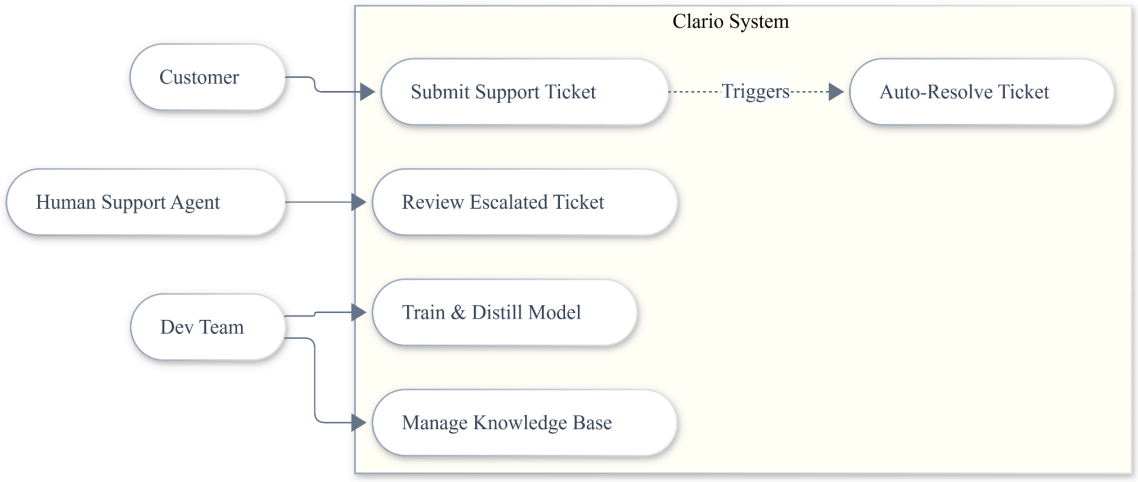


Figure 1: Clario Use-Case Diagram. Customers submit tickets (UC-01) which automatically trigger the multi-agent triage pipeline (UC-02). When the pipeline cannot resolve a ticket autonomously, it extends to human review (UC-03). The Dev Team manages the knowledge base and model lifecycle.

4.1 Use-Case Realizations

This section illustrates how the software actually works by providing selected use-case realizations, explaining how the various design model elements contribute to their functionality.

Here are the five tables from this shortened version, ready to paste into Google Docs:

UC-01: Submit Support Ticket

Field	Description
Use Case Name	Submit Support Ticket
Actor	Customer
Description	A customer submits a new support ticket through the web interface. The system persists the ticket and asynchronously triggers the AI triage pipeline.

Preconditions	Customer is authenticated (JWT token valid). The Spring Boot gateway and ML Sidecar are running.
Main Flow	1. Customer fills out the ticket form in the Next.js UI and clicks Submit. 2. UI sends POST /tickets with JWT bearer token to the Spring Boot Gateway. 3. Gateway authenticates the JWT, validates the request body. 4. Gateway persists the ticket to PostgreSQL with status = received. 5. Gateway returns 202 Accepted with ticket_id to the UI. 6. Gateway dispatches POST /process_ticket to the ML Sidecar on a separate background thread (non-blocking).
Successful End Condition	Ticket is persisted in PostgreSQL. Customer sees a confirmation message with their ticket_id. The ML pipeline begins executing asynchronously.
Fail End Condition	If JWT is invalid -> 401 Unauthorized returned; ticket not persisted. If ML Sidecar is unreachable -> ticket remains in received state; retried or escalated.
Extensions	If the customer is not logged in, they are redirected to the login page.

UC-02: Auto-Triage and Respond

Field	Description
Use Case Name	Auto-Triage and Respond
Actor	System (LangGraph ML Sidecar)
Description	The LangGraph pipeline autonomously processes a ticket: masks PII, classifies it, routes to a specialist agent, generates a grounded RAG draft, validates it, and either auto-sends or escalates.
Preconditions	Ticket exists with status = received. ML Sidecar has received the ticket payload.
Main Flow	1. Cache Check -search ChromaDB for a near-duplicate resolved ticket. If found, reuse the cached draft. 2. SurrogateShield -mask PII; store mapping in ShadowMap. 3. Classification -categorise ticket by type, priority, and sentiment. 4. Routing -dispatch to Technical Agent, Billing Agent, or both. 5. RAG Draft -retrieve relevant KB chunks and generate a grounded response. 6. Validation -run policy checks and a gated LLM Judge. Retry or reroute if needed.

	7. Resolve and Handoff -restore PII and package the final result; update PostgreSQL.
Success	Ticket resolved automatically; response stored and embedded as precedent.
Failure	All retry/reroute attempts exhausted -> ticket escalated to human review.

UC-03: Review Escalated Ticket

Field	Description
Actor	Human Support Agent
Description	A Human Agent reviews an escalated ticket, inspects the AI-generated handoff package, and approves, edits, or rejects the draft. Optimistic locking prevents concurrent overwrites.
Preconditions	Ticket has status = escalated. Agent is authenticated with ROLE_AGENT.
Main Flow	1. Agent opens the Review Screen and views the list of escalated tickets. 2. Agent opens a ticket -sees AI draft, classification, RAG sources, and validation reasoning. 3. Agent edits (if needed) and submits a decision via PATCH /tickets/{id}/review. 4. Gateway performs an atomic version check in PostgreSQL and returns 200 OK on success.
Success	Ticket closed. Customer receives the final human-reviewed response.
Failure	Two agents submit at the same time -> second receives 409 Conflict; their edits are preserved in draftBuffer for re-application.

UC-04: Manage Knowledge Base

Field	Description
Actor	Dev Team
Description	The Dev Team updates knowledge base Markdown files and re-indexes them into ChromaDB for RAG retrieval.
Preconditions	Docker Compose stack is running. ChromaDB is healthy.

Main Flow	1. Edit or add .md files in kb_documents/technical/ or billing/. 2. Run python build_index.py. 3. Script chunks, embeds, and upserts to ChromaDB (idempotent).
Success	New knowledge is available for RAG retrieval on the next ticket.
Failure	ChromaDB unreachable -> script errors; existing index is unchanged.

UC-05: Train and Distill Model

Field	Description
Actor	Dev Team
Description	The Dev Team runs the offline ML pipeline: cleaning data, distilling labels via Gemini, training a QLoRA adapter, and evaluating it on the held-out test set.
Preconditions	Raw Kaggle dataset downloaded. Gemini API key configured. Kaggle/Colab GPU available.
Main Flow	1. pii_clean.py -one-time offline PII masking. 2. gemini_labeler.py -async batch labelling with Gemini. 3. train_lora.py -QLoRA fine-tuning on Mistral-7B. 4. evaluate.py -evaluate on locked test.parquet (once only). 5. Register approved adapter in models/registry.json.
Success	A QLoRA adapter with acceptable accuracy is registered and deployed to the ML Sidecar.
Failure	Model below threshold -> rollback to previous version; Gemini Flash stand-in continues.

5. Logical View

5.1 Overview

The logical design is decomposed into three major tiers: the API Gateway (Spring Boot), the ML Sidecar (Python/FastAPI), and the Frontend Application (Next.js).

5.2 Architecturally Significant Design Packages

Core Domain Model (UML Class Diagram)

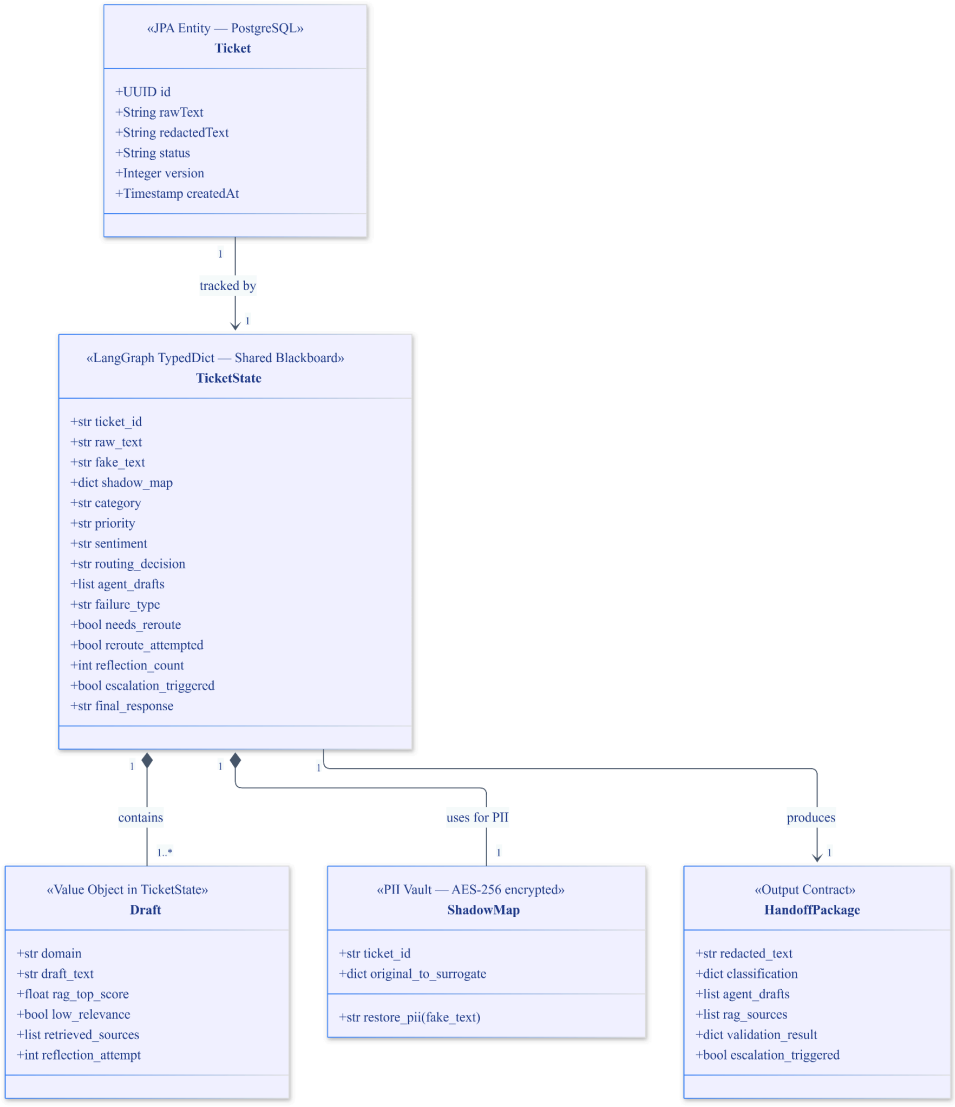


Figure 2: Core domain class diagram. The `Ticket` entity lives in PostgreSQL (Spring Boot). The `TicketState` TypedDict is the shared LangGraph blackboard. `ShadowMap` maintains the PII reversal vault. `HandoffPackage` is the final output contract between the ML Sidecar and the Human Review screen.

System Context (C1)

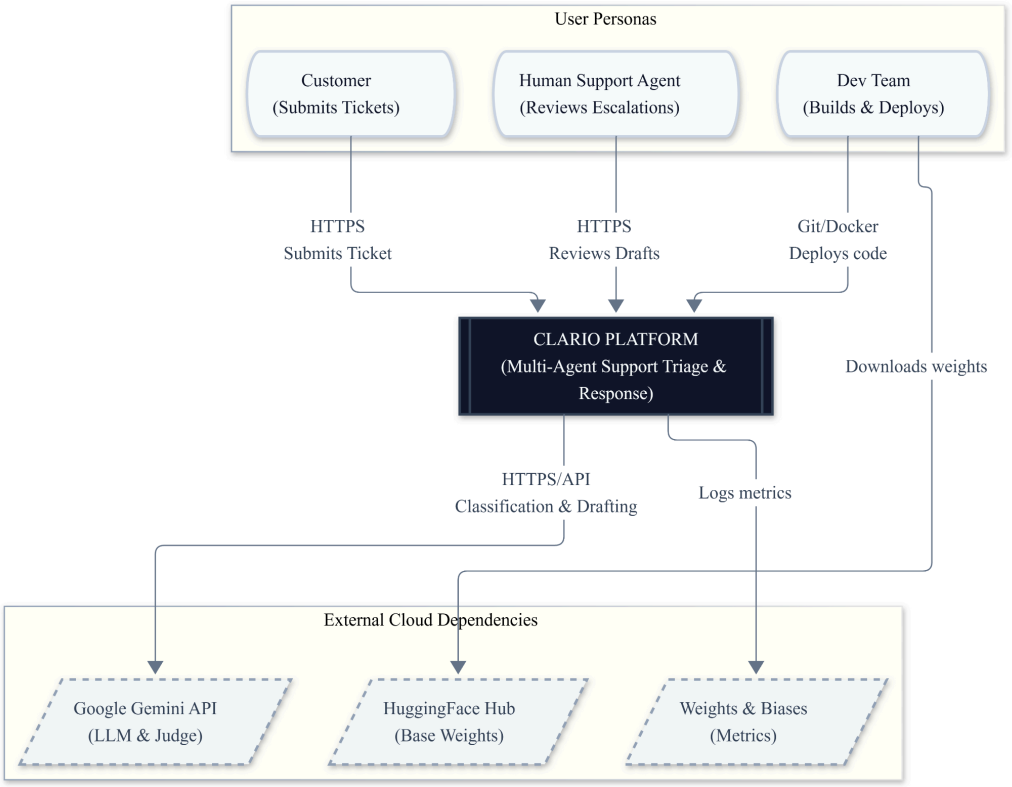


Figure 3: C1 System Context. Clario is the central platform. Customers and Human Agents interact over HTTPS. The Dev Team deploys via Git and Docker. Google Gemini is the primary external LLM provider used for classification, drafting, and judging.

Container Architecture (C2)

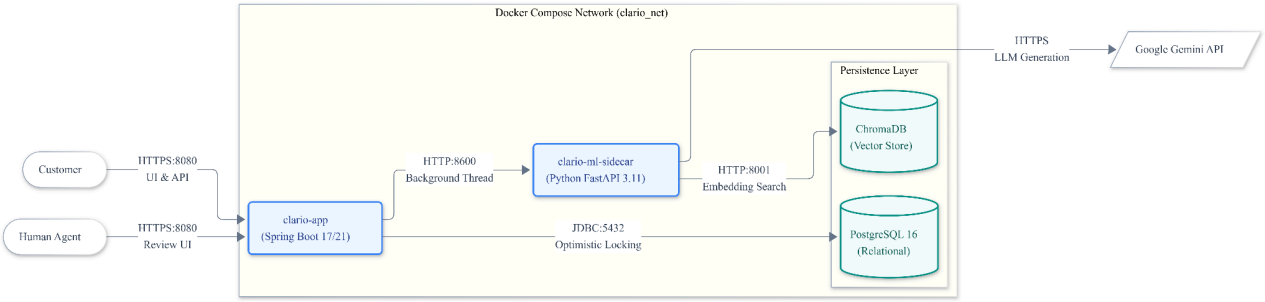
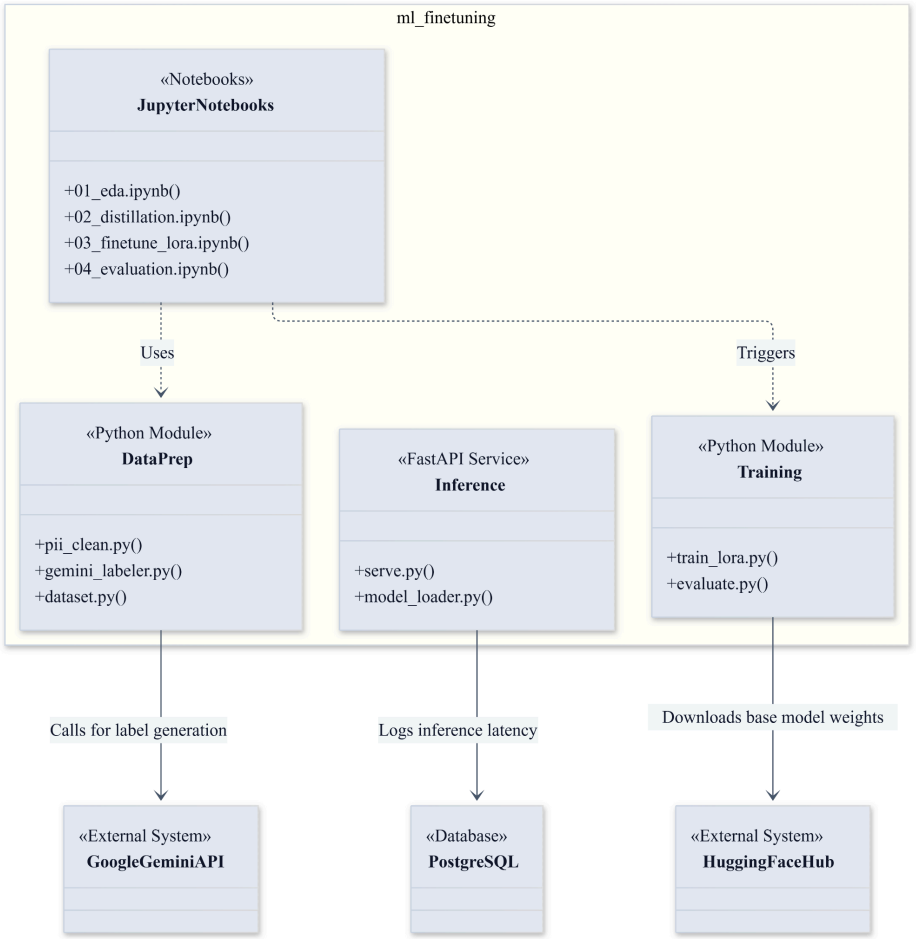


Figure 4: C2 Container Diagram. All four containers run in the same Docker Compose network. The Spring Boot gateway is the only externally exposed container (port 8080). The ML Sidecar (port 8600) is internal-only, invoked asynchronously by the gateway.

Service	Internal Port	Host-Mapped Port
Clario-app (Spring Boot + UI)	8080	8080
Clario-ml-sidecar	8600	8600
chromaDB	8000	8001
postgres	5432	5432

Component Architecture & LangGraph State Machine

5.3 ML Fine-Tuning Service



5.4 ML Sidecar (Python ML Sidecar)

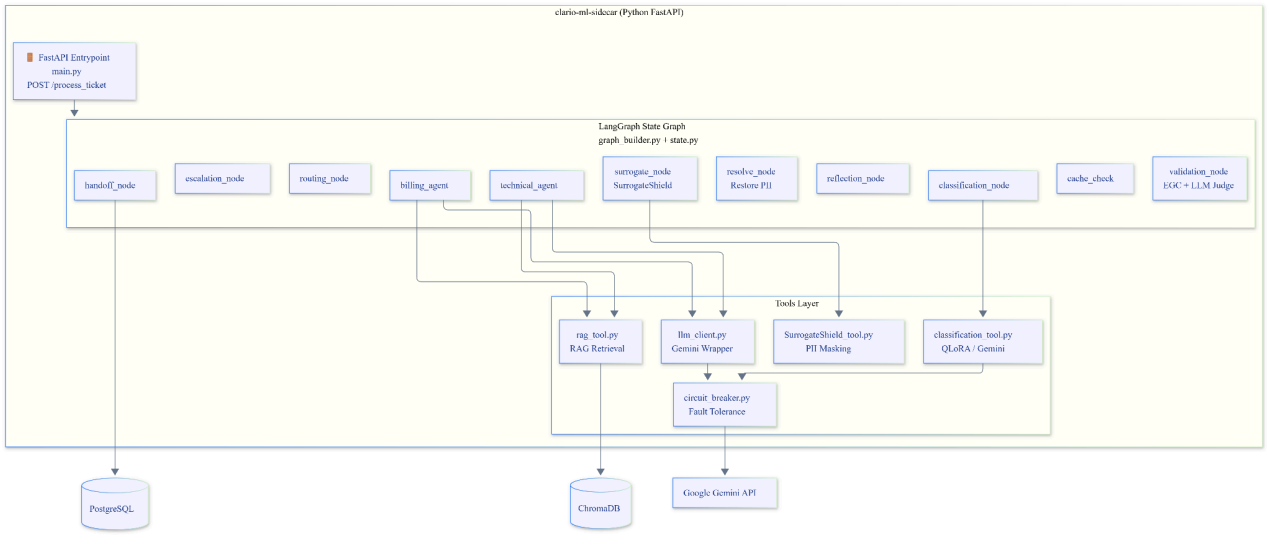


Figure 6: C3 ML Sidecar component diagram. The FastAPI entrypoint invokes the compiled LangGraph. Each graph node delegates to the appropriate tool. All external LLM calls pass through the circuit breaker for fault tolerance.

5.5 API Gateway Service (Spring Boot Monolith)

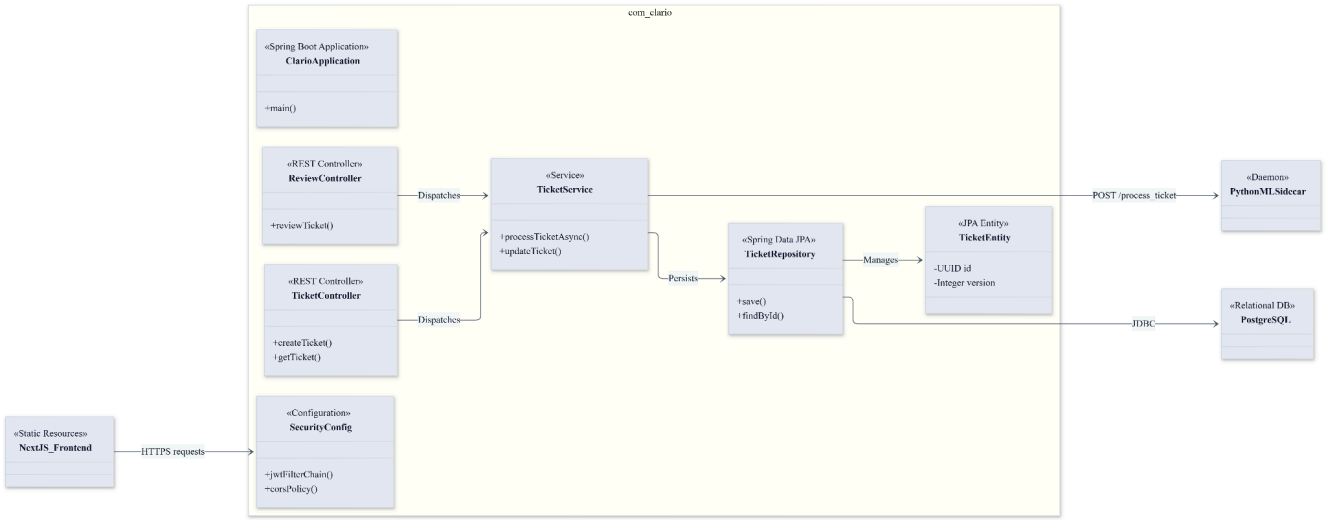


Figure 7: This diagram shows the internal structure of the Java Spring Boot backend, which acts as the main gateway for the Clario platform.

How it works:
Security & Entry: *Requests from the Next.js frontend hit the SecurityConfig first, which checks if the user's JWT token is valid.*
Controllers:*The TicketController handles customers submitting or fetching tickets.*
The ReviewController handles human agents reviewing and approving escalated tickets.

Service Layer: *Both controllers pass data to the TicketService. This is where the core business logic lives. Crucially, it has a method processTicketAsync() that fires off the ticket to the Python ML Sidecar in the background so the customer doesn't have to wait.*
Database Layer: *The TicketService uses the TicketRepository to talk to the PostgreSQL database. This repository manages the TicketEntity, which represents a single ticket in the database. Notice the version field on the TicketEntity, this is what prevents two human agents from accidentally saving over each other's work at the same time (optimistic locking).*

5.6 Frontend Application

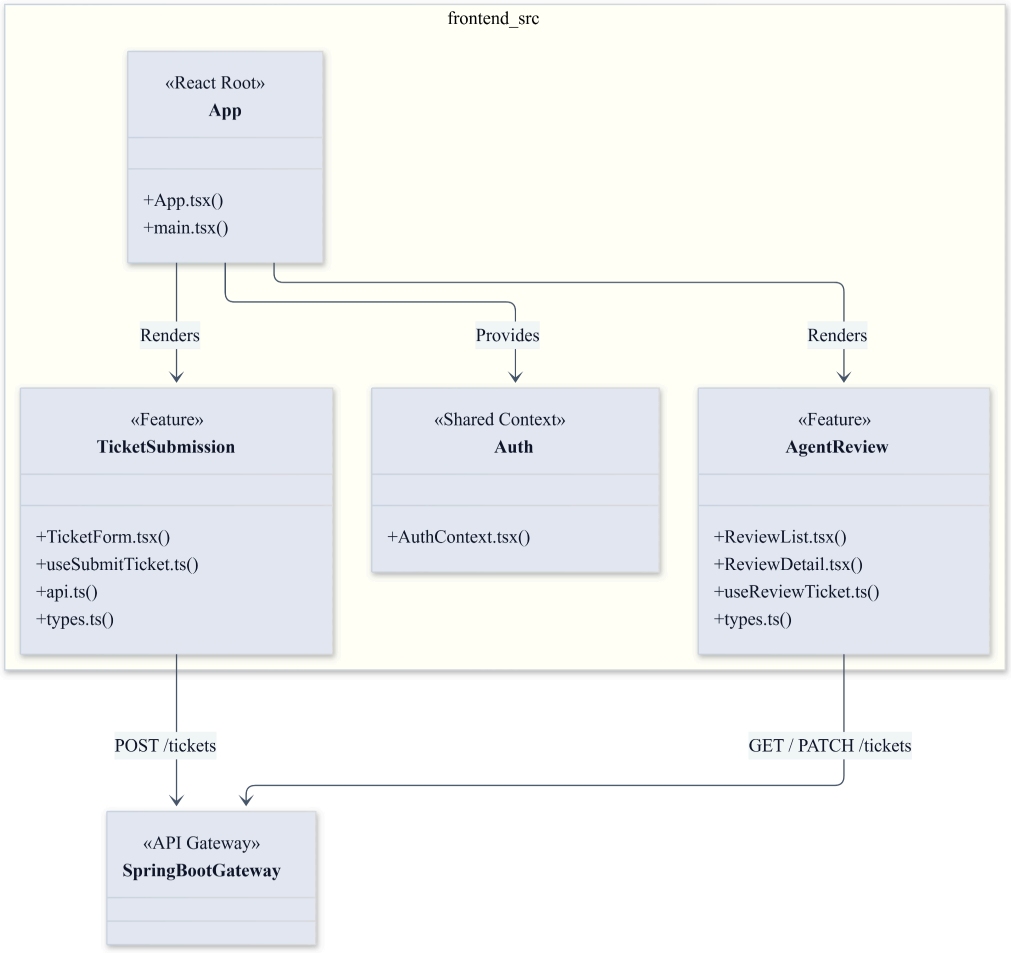


Figure 8: The Next.js frontend is split into two feature modules. TicketSubmission (for customers) and AgentReview (for human agents) both mounted under a single App root with shared Auth context. All API calls flow outward to the Spring Boot Gateway via REST.

5.7 Vector Store Service

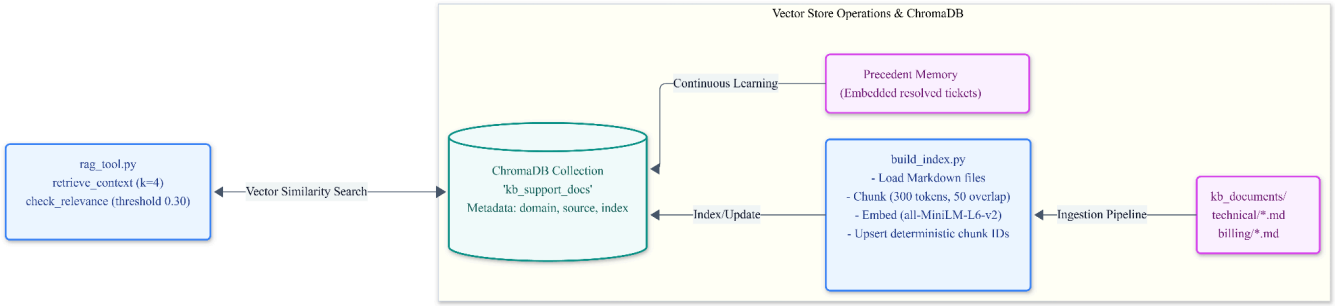


Figure 9: Shows how ChromaDB is built and used. Offline: knowledge base Markdown files are chunked, embedded, and indexed into ChromaDB. At runtime: rag_tool.py queries ChromaDB by vector similarity to retrieve the most relevant knowledge chunks for each ticket. Resolved tickets are continuously fed back in as precedent memory for future cache hits.

5.8 Relational Database

See [Section 9 -Database Schema (ERD)] for the full ERD diagram.

LangGraph State Machine

5.9 TicketState -Shared Blackboard

The `TicketState` TypedDict is the single shared contract between all nodes. The `**failure_type**` field is the sole routing signal used by `graph_builder`'s conditional edges.

Failure_type value	Next Step
“none”	Proceed to escalation check (normal path)
“misroute”	Reroute (flip domain) - capped at 1 attempt
“quality” or “policy”	Bounded reflection retry to escalate if cap reached
“dependancy_failure”	Immediate escalation (external call failed after retries)

5.10 LangGraph State Graph

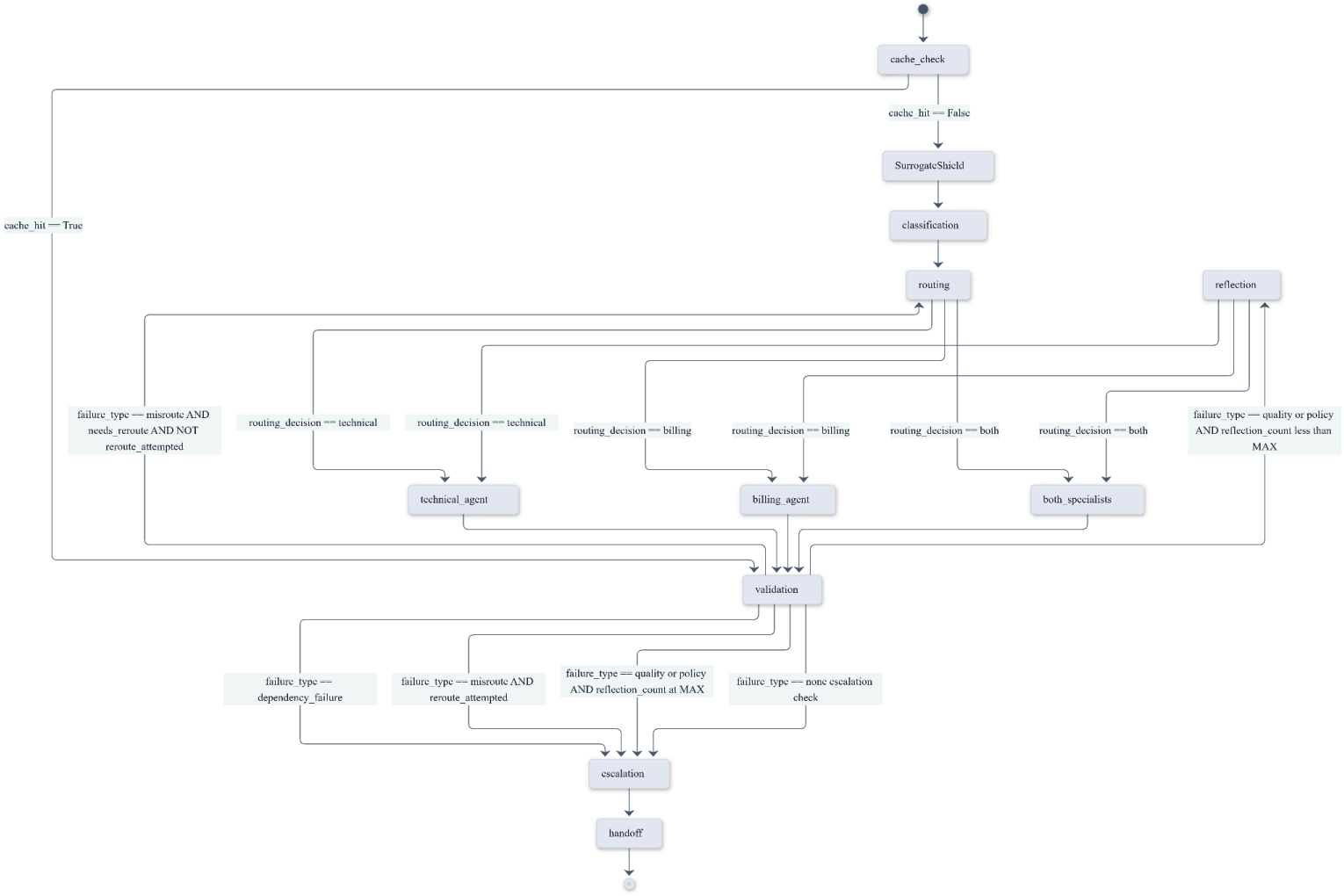


Figure 10: This diagram shows the exact decision-making logic of the ML Sidecar as it processes a ticket.

How it flows:

Start & Prep: *The system checks if it has seen this ticket before (cache_check). If not, it removes personal info (SurrogateShield), figures out what the ticket is about (classification), and decides who should handle it (routing).*

Drafting: *The ticket is sent to either the technical_agent, the billing_agent, or both_specialists to draft a response using the knowledge base.*

The Judge (Validation): *Every draft must pass the validation node. The judge checks if the response is safe, accurate, and on-topic.*

Self-Correction (The Loops):*If the judge says the draft is poor quality, it loops back to reflection to try again (up to a limit). If the judge realizes the ticket was sent to the wrong department, it loops back to routing to flip the domain.*

Resolution: *If the draft is perfect (or if it fails too many times and must be escalated), it goes to escalation (to check if human review is needed), and finally to handoff where personal info is restored and the ticket is sent back to the database.*

5.11 Reroute Logic

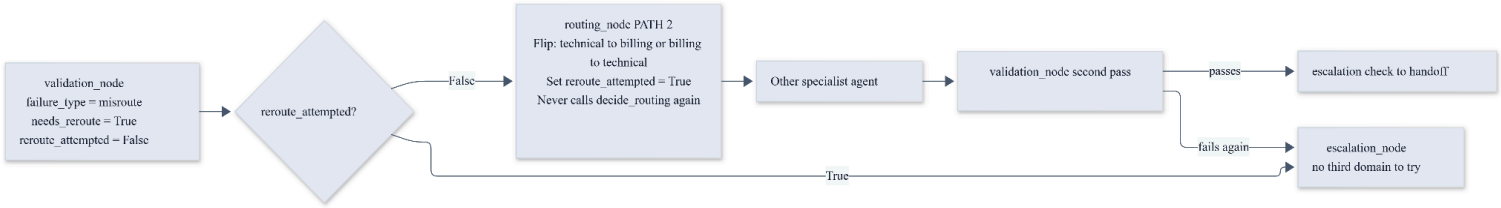


Figure 11: This diagram explains a specific fail-safe mechanism in the AI's routing process.
The Problem it Solves: *Sometimes the AI classifier might send a ticket to the technical department when it really belongs to billing. When the technical agent tries to answer it, the Judge node catches the mistake and flags it as a misroute.*

How it works:
When a misroute happens, the system first checks if it has already tried to reroute this ticket (reroute_attempted?)
If False (first time): *Instead of asking the classifier to guess again (which might result in the same wrong answer), it simply flips the domain. If it was technical, it forces it to billing. It marks reroute_attempted = True and sends it to the other agent.*
Second Pass: *The new agent drafts a response and it goes back to the Judge. If it passes, great!*
If True (already tried)*: If it fails the Judge again, the system knows there's no third department to try. To prevent an infinite loop, it immediately stops and sends the ticket to human escalation.*

6. Process View

This section describes Clario's system decomposition into processes (threads of control) and documents how these processes communicate.

Concurrency model: The Spring Boot gateway runs the LangGraph invocation on a dedicated background thread (not the HTTP request thread). This ensures the customer receives the `202 Accepted` response in under 200ms regardless of LLM API latency. The ML Sidecar itself is single-threaded per ticket execution but can handle multiple tickets concurrently via FastAPI's async/await.

Ticket Lifecycle Activity Diagram (LangGraph Flow)

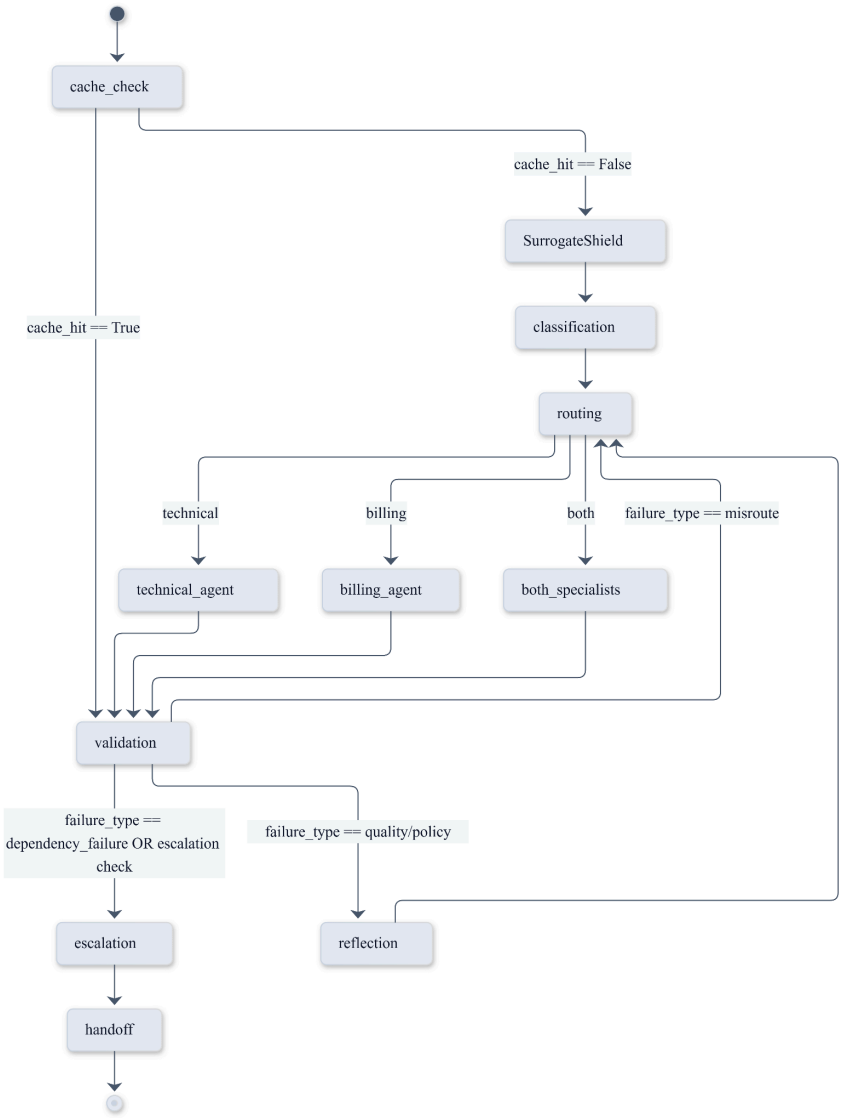


Figure 12: Full ticket lifecycle activity diagram. The LangGraph state machine processes tickets autonomously. Reflection and rerouting are bounded to prevent infinite loops. All paths converge at the RestorePII and HandoffNode steps.

***Data Flow -* [Link to View]**

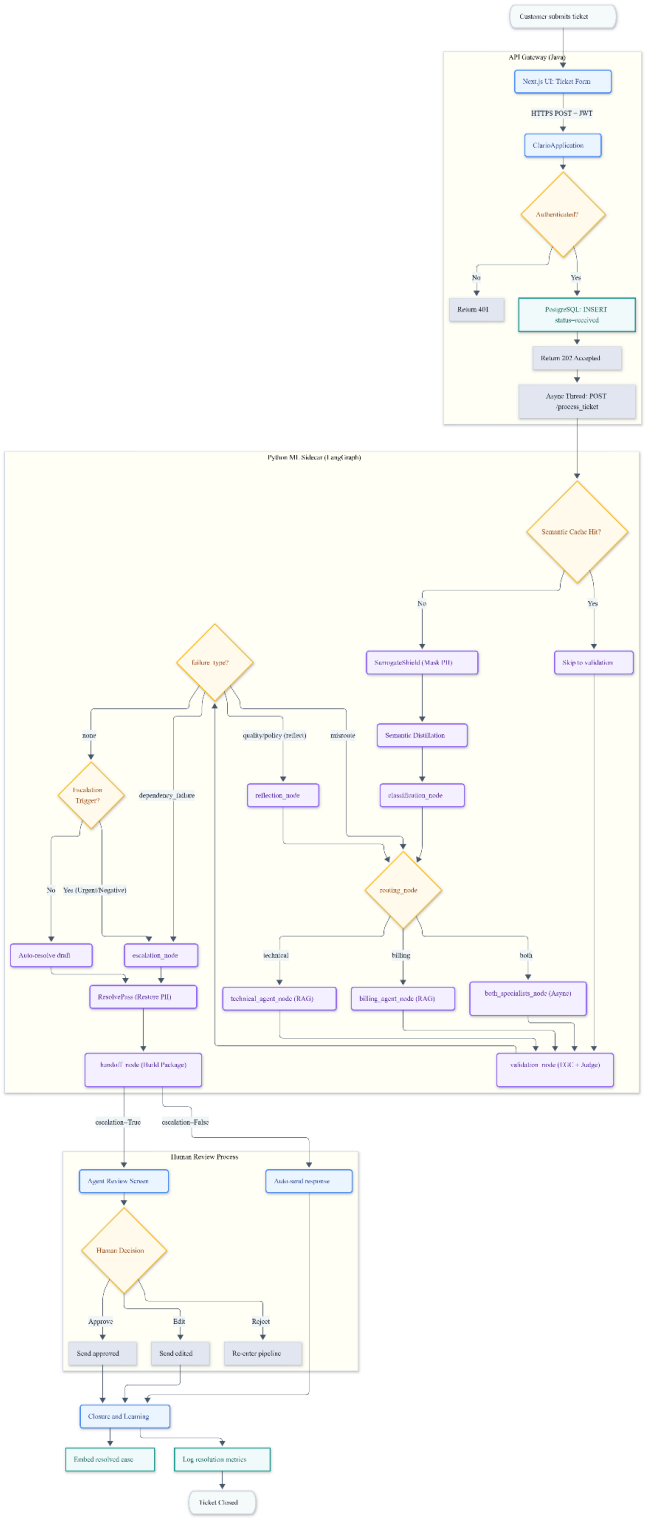


Figure 13: This diagram shows the internal structure of the Python ML Sidecar (clario-ml-sidecar).

How it works: *The system receives a ticket via the FastAPI Entrypoint at the top left. From there, it passes into the LangGraph State Graph (the large middle box), which acts as the brain. The ticket moves step-by-step through the various nodes (like caching, masking PII, routing, the specialist agents, and validation).*

Whenever a node needs to talk to the outside world, it uses the Tools Layer (the box below it) to fetch knowledge from ChromaDB, mask PII, or make external API calls to Google Gemini. Finally, the processed ticket is saved back to PostgreSQL at the end of the graph execution.

6.1 Happy Path (Auto-Send)

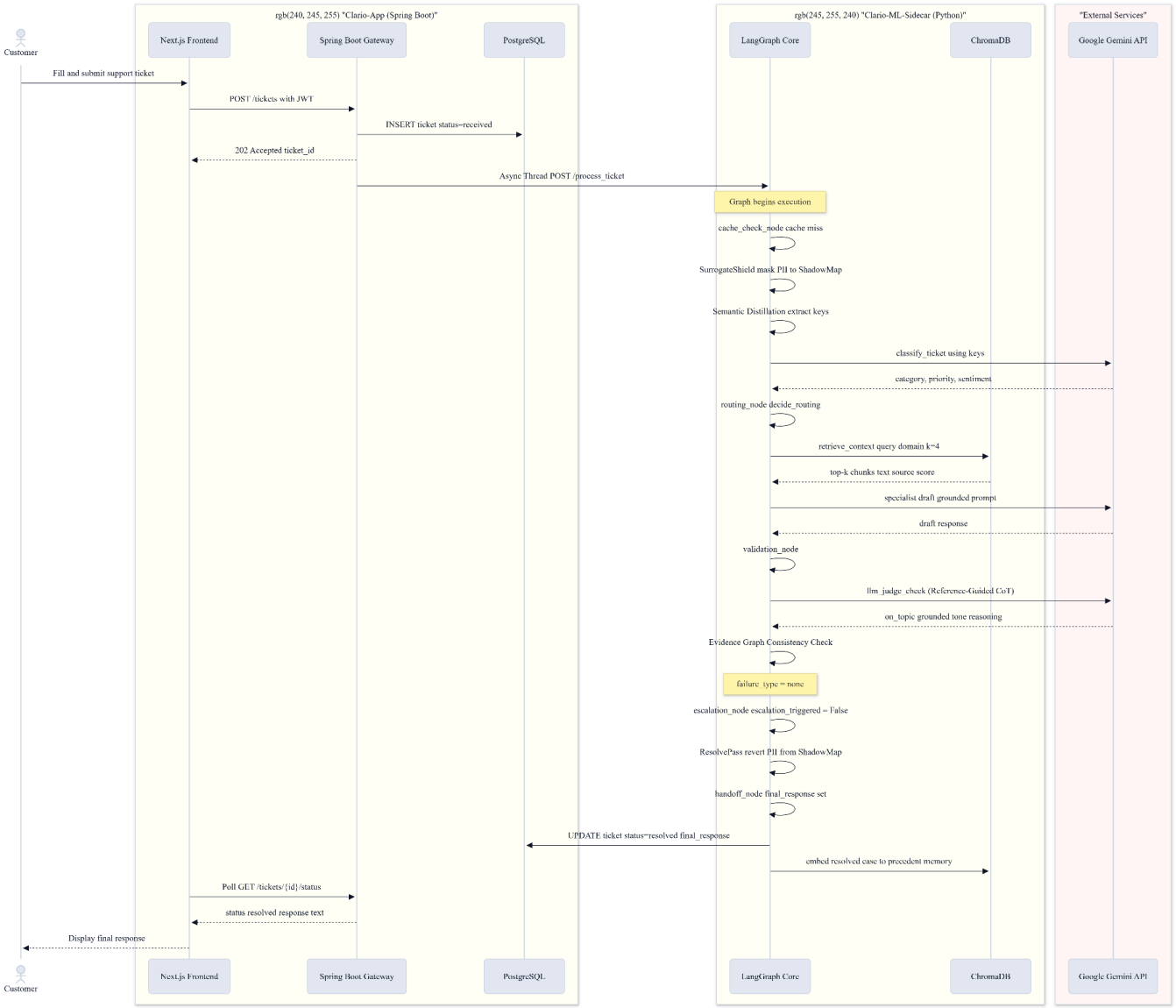


Figure 14: Happy path sequence diagram. A standard ticket is processed end-to-end without escalation. The customer receives a `202 Accepted` immediately; the AI pipeline executes asynchronously.

6.2 Escalation to Human Review

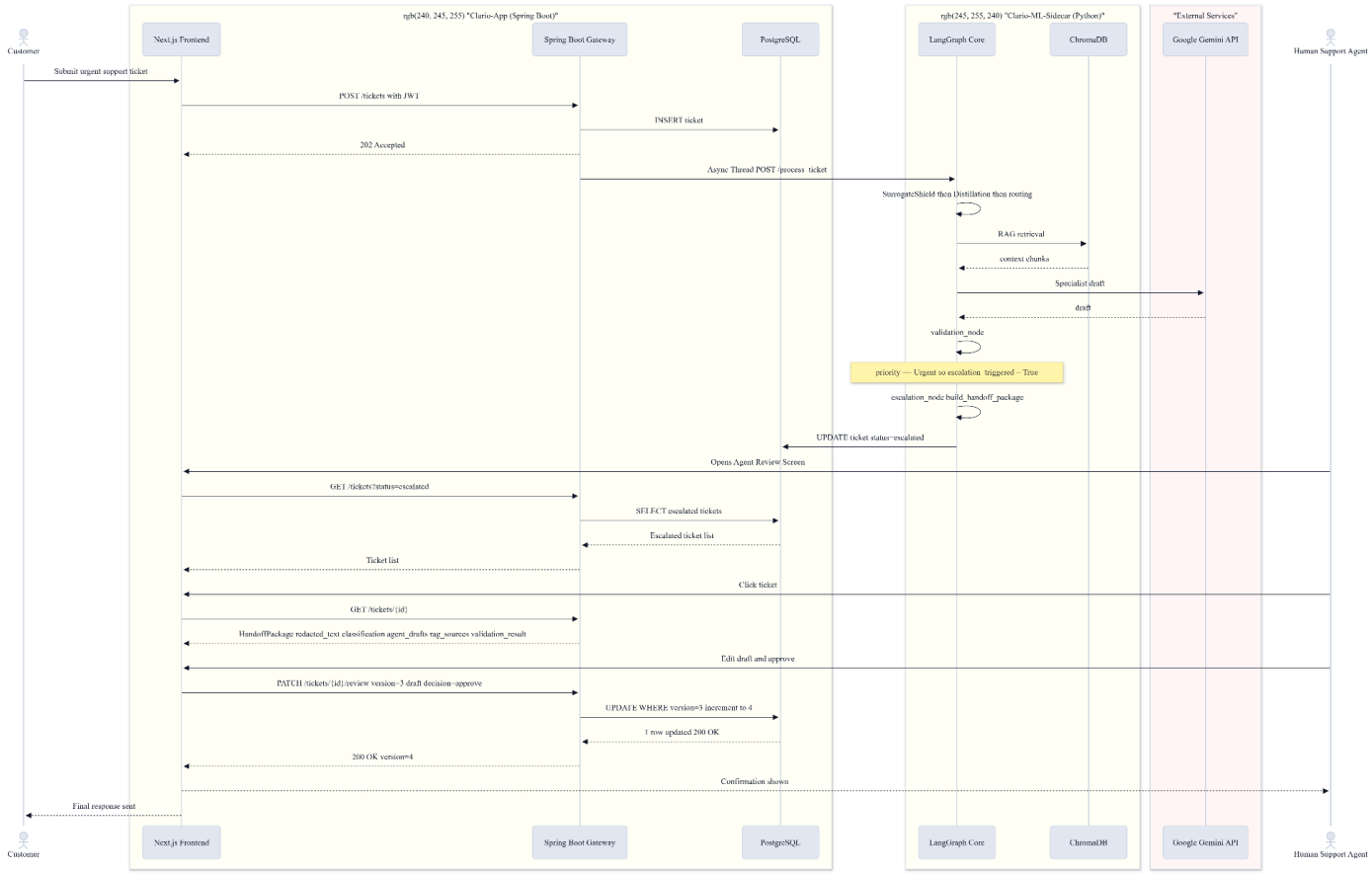


Figure 15: This diagram shows the flow when the AI pipeline cannot resolve a ticket automatically and has to escalate it to a human.

How it works (Top to Bottom):

Submission*: A customer submits an urgent ticket. The Spring Boot Gateway immediately saves it as received and returns a 202 Accepted to the customer.*

AI Processing*: The Gateway triggers the ML Sidecar in the background. The LangGraph engine tries to process it (fetching RAG context and asking Gemini for a draft), but during the validation step, it realizes the ticket is urgent (or fails quality checks), so it sets status = escalated in the database.*

Human Review: *A Human Support Agent logs in and opens their Review Screen. They click on the escalated ticket and receive a complete "Handoff Package" (which includes the AI's draft, the classification, and the RAG sources it used).*

Resolution*: The human agent edits the draft and clicks Approve. The Gateway updates the ticket in PostgreSQL (incrementing the version to prevent conflicts). The customer then receives the final, human-approved response.*

6.3 Optimistic Locking -409 Conflict

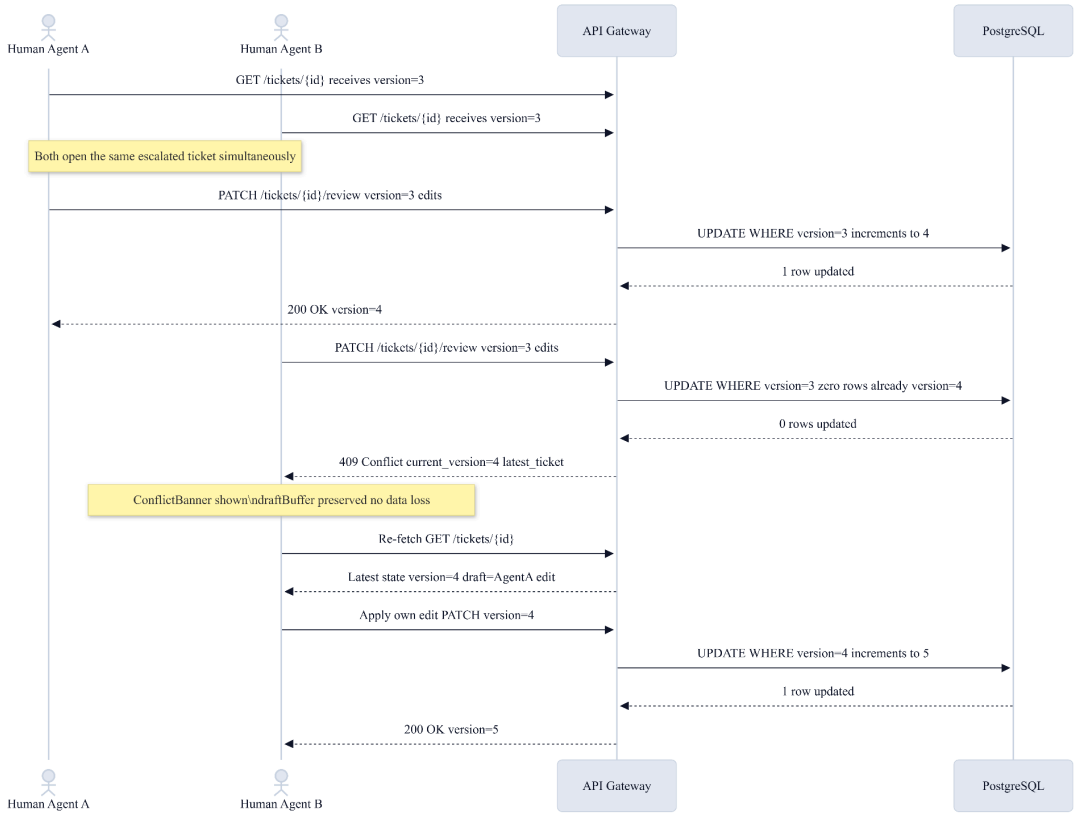
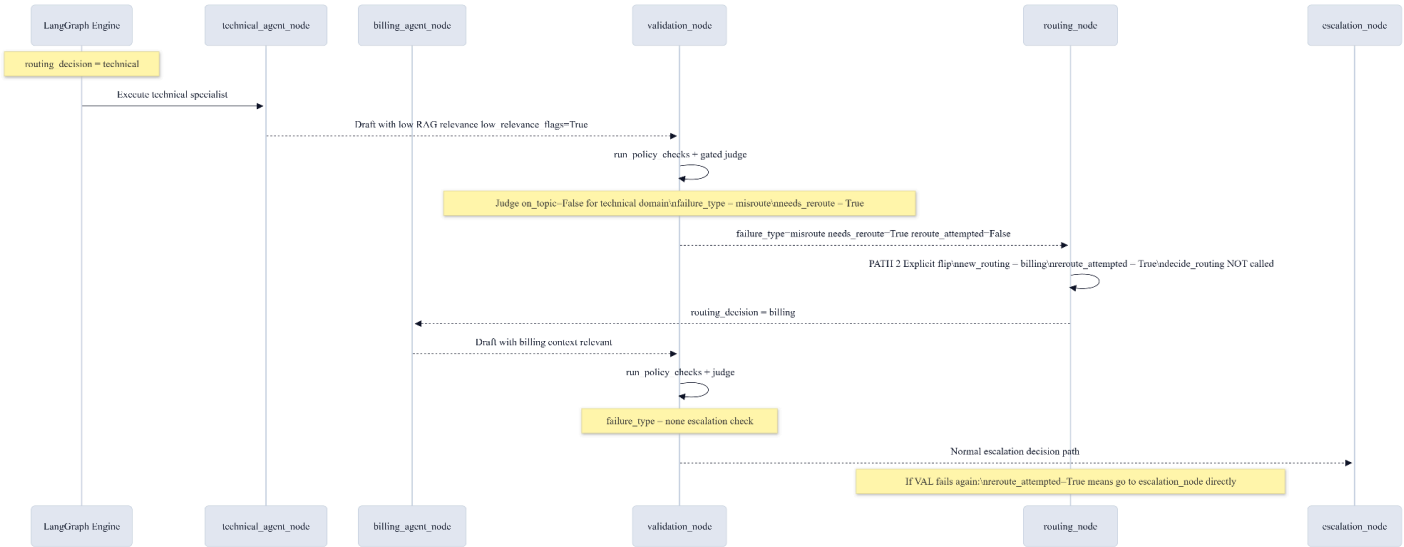


Figure 16: Optimistic locking sequence. Two agents cannot silently overwrite each other's edits. The second agent receives a 409 Conflict, their edits are preserved in the UI's `draftBuffer`, and they can cleanly re-apply their changes after re-fetching.

6.4 Reroute Flow



7. Deployment View

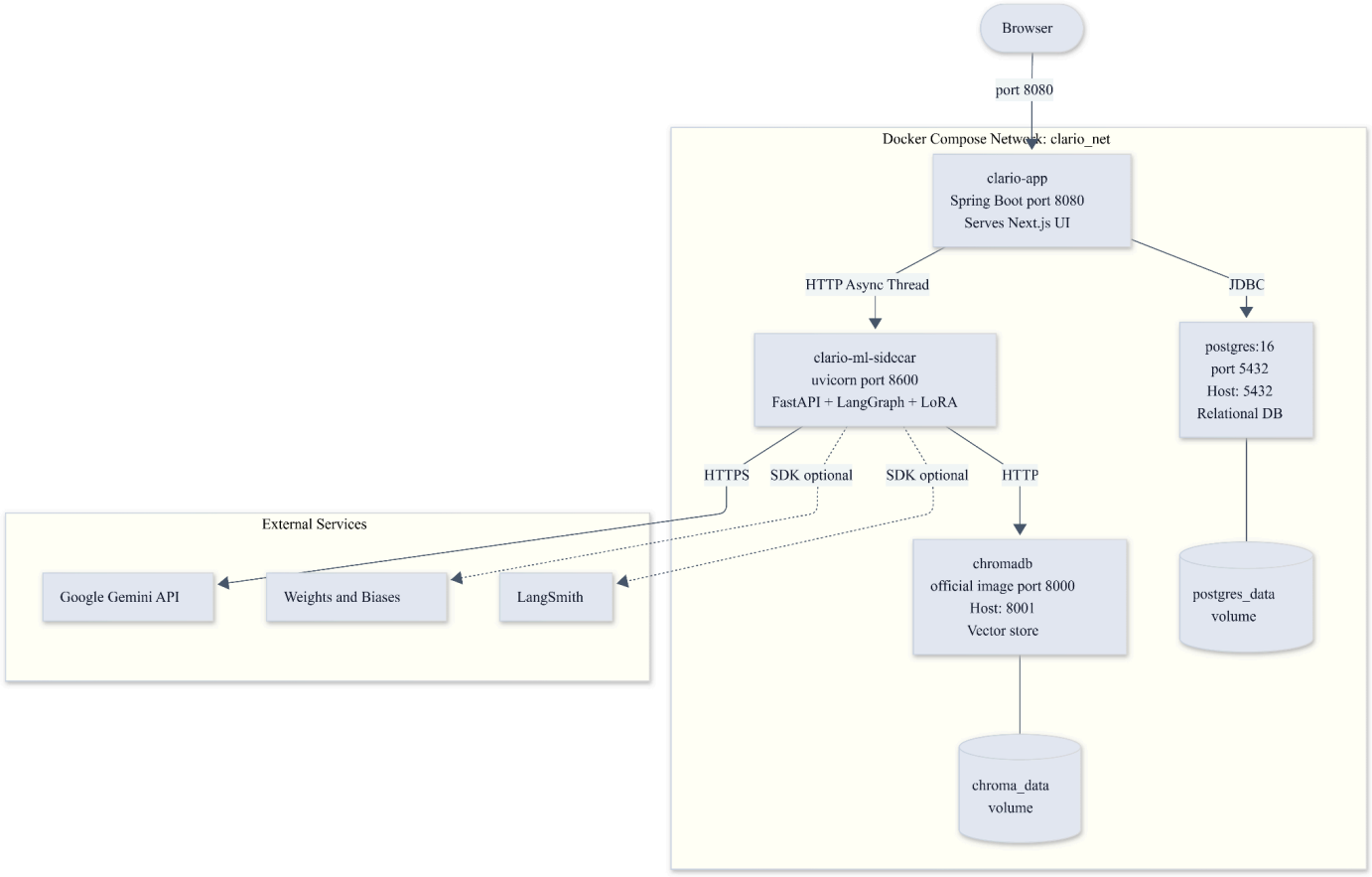


Figure 18: Deployment diagram. All four containers run on a single host machine within the `clario_net` Docker bridge network. Only the Spring Boot container is externally exposed. Named volumes ensure ChromaDB and PostgreSQL data persist across container restarts.

8. Implementation View

8.1 Overview

The software implementation is organized into distinct layer packages representing the Frontend, API Gateway, ML Sidecar, and ML Fine-Tuning pipelines.

8.2 Layers

8.2.1 ML Sidecar Packages - Link

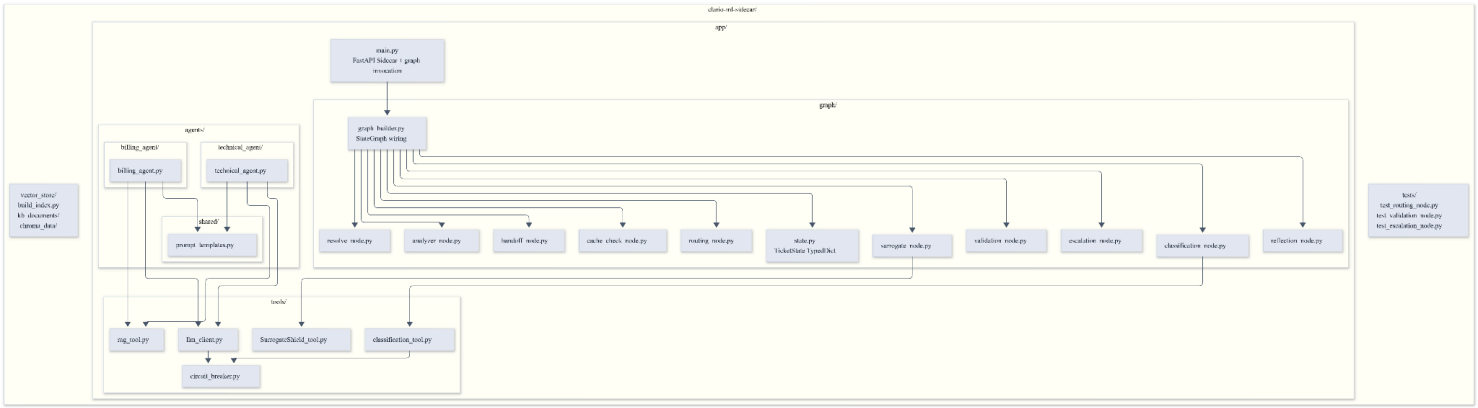


Figure 19: This diagram shows how the actual code folders and files are organized inside the clario-ml-sidecar component.

How it's organized:
Entry Point (main.py): *This is the top-level file that starts the FastAPI server.*
graph/ folder: *Contains all the individual steps (nodes) of the AI pipeline (like routing_node.py, validation_node.py, etc.) and the wireframe that connects them (graph_builder.py).*
agents/ folder: *Contains the specific prompts and logic for the two specialist AI agents (technical_agent and billing_agent).*
tools/ folder: *Contains the utility scripts that the graph and agents use to do actual work, like masking PII (SurrogateShield_tool.py), talking to the database (rag_tool.py), or calling external APIs safely (circuit_breaker.py).*
Outside the main app: *There are separate folders for tests/ and the vector_store/ (which holds the knowledge base markdown files and the script to build the ChromaDB index).*

8.2.2 API Gateway Packages

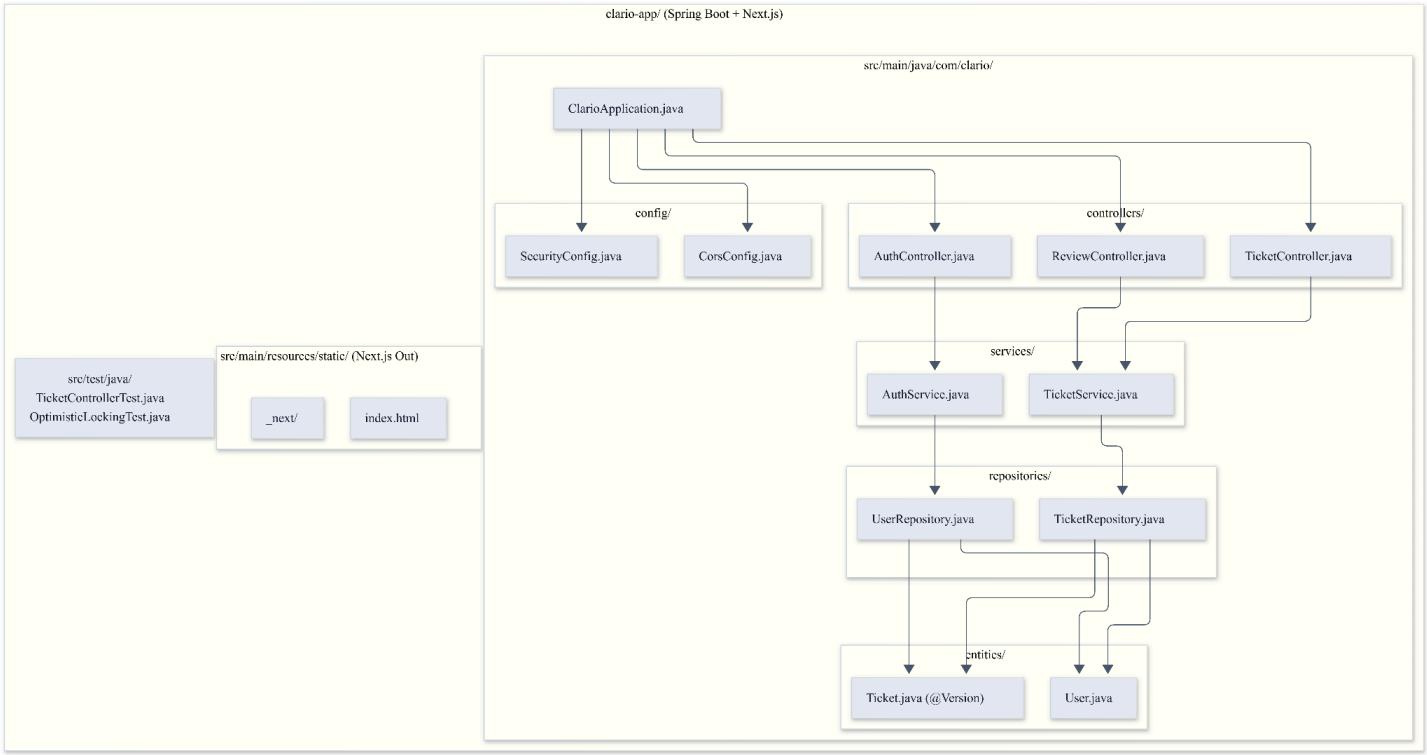


Figure 20: Spring Boot API Gateway package diagram. The `@Version` annotation on `Ticket.java` is the key design element enabling optimistic locking. `TicketService.java` dispatches asynchronously to the ML Sidecar to keep the HTTP request thread unblocked.

8.2.3 ML Fine-Tuning Packages

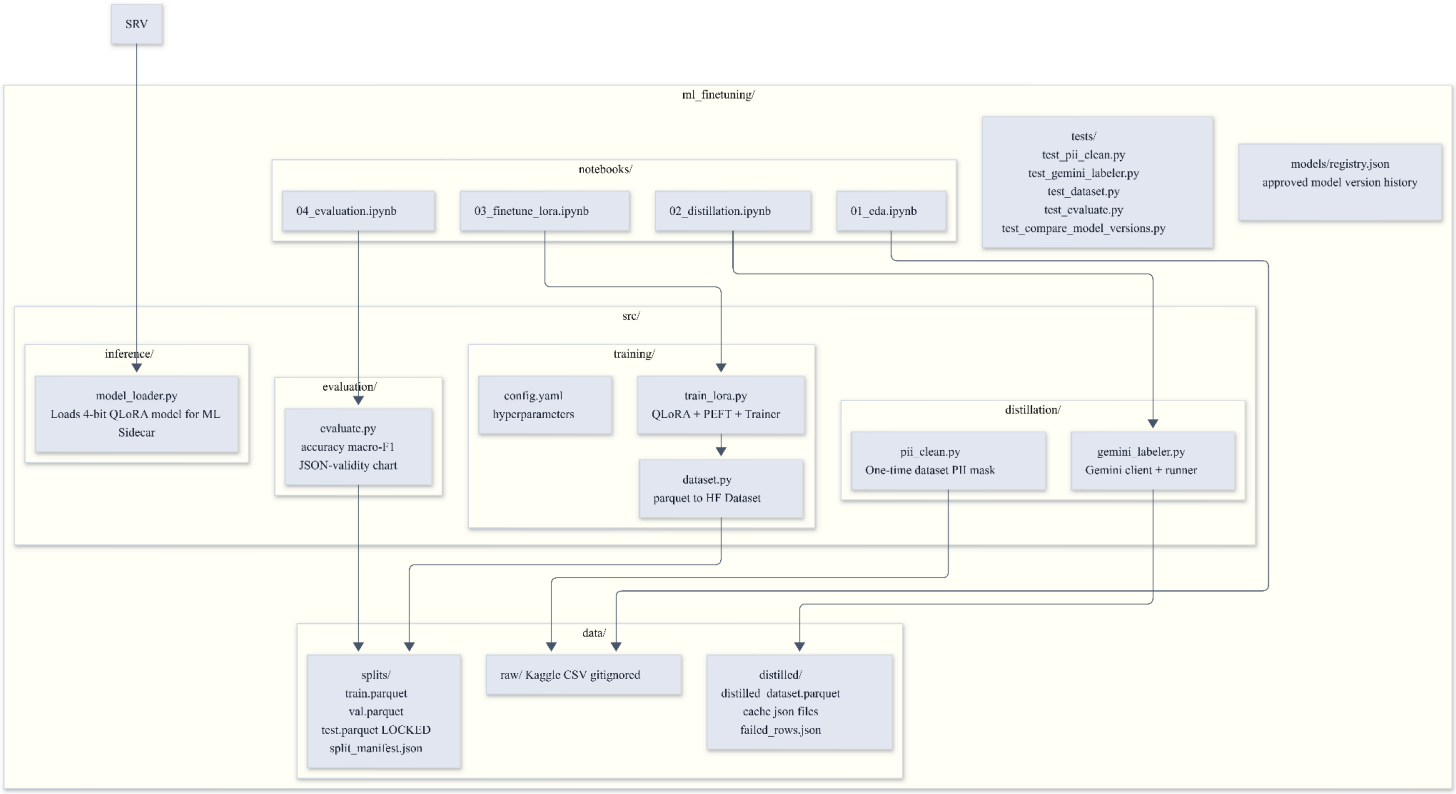


Figure 21: ML Fine-Tuning package diagram. The pipeline flows left to right: raw data -> PII cleaning -> distillation -> QLoRA training -> evaluation -> model registry. The `test.parquet` split is locked and evaluated exactly once.

8.2.4 Frontend Feature Packages

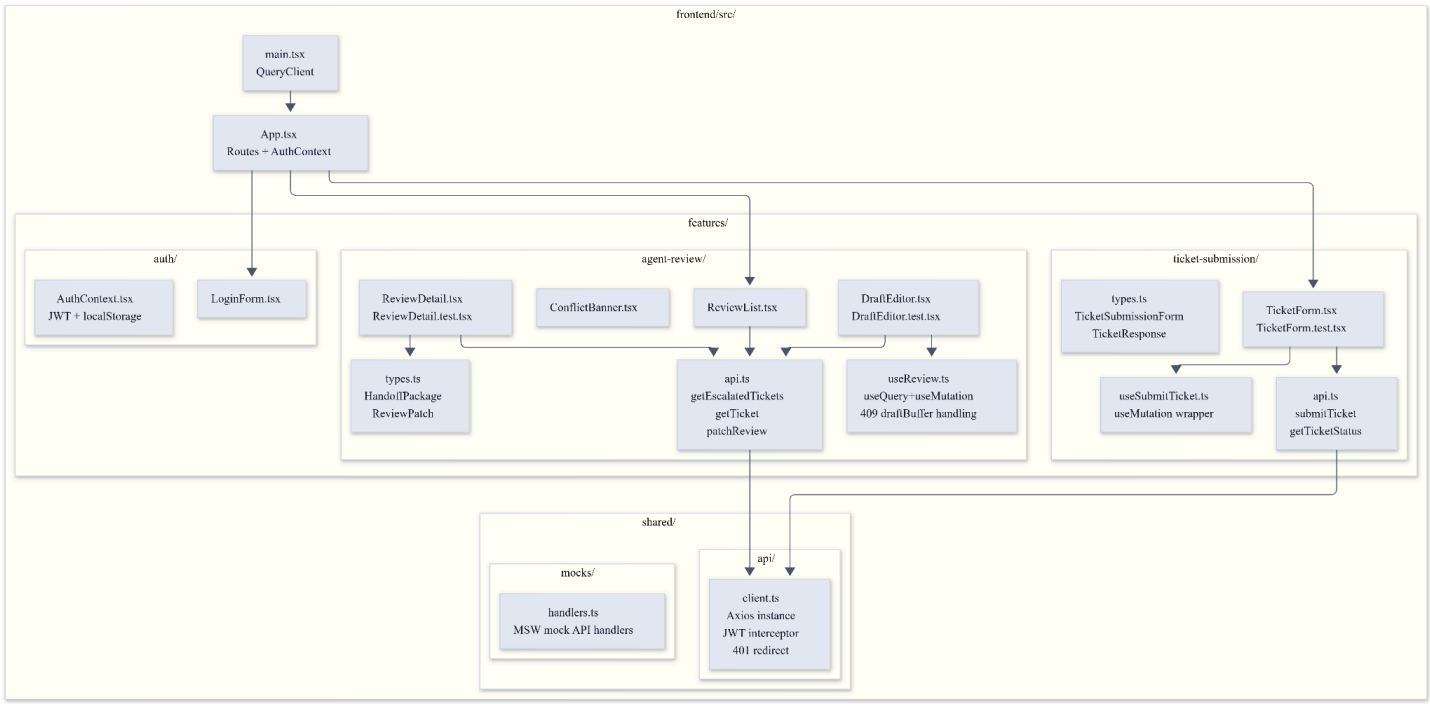


Figure 22: Next.js frontend package diagram. The feature-based structure keeps ticket submission and agent review concerns cleanly separated. `ConflictBanner.tsx` and the `draftBuffer` in `useReview.ts` are the key UI elements for handling 409 Conflict responses.

9. Data View (This may change during the implementations)

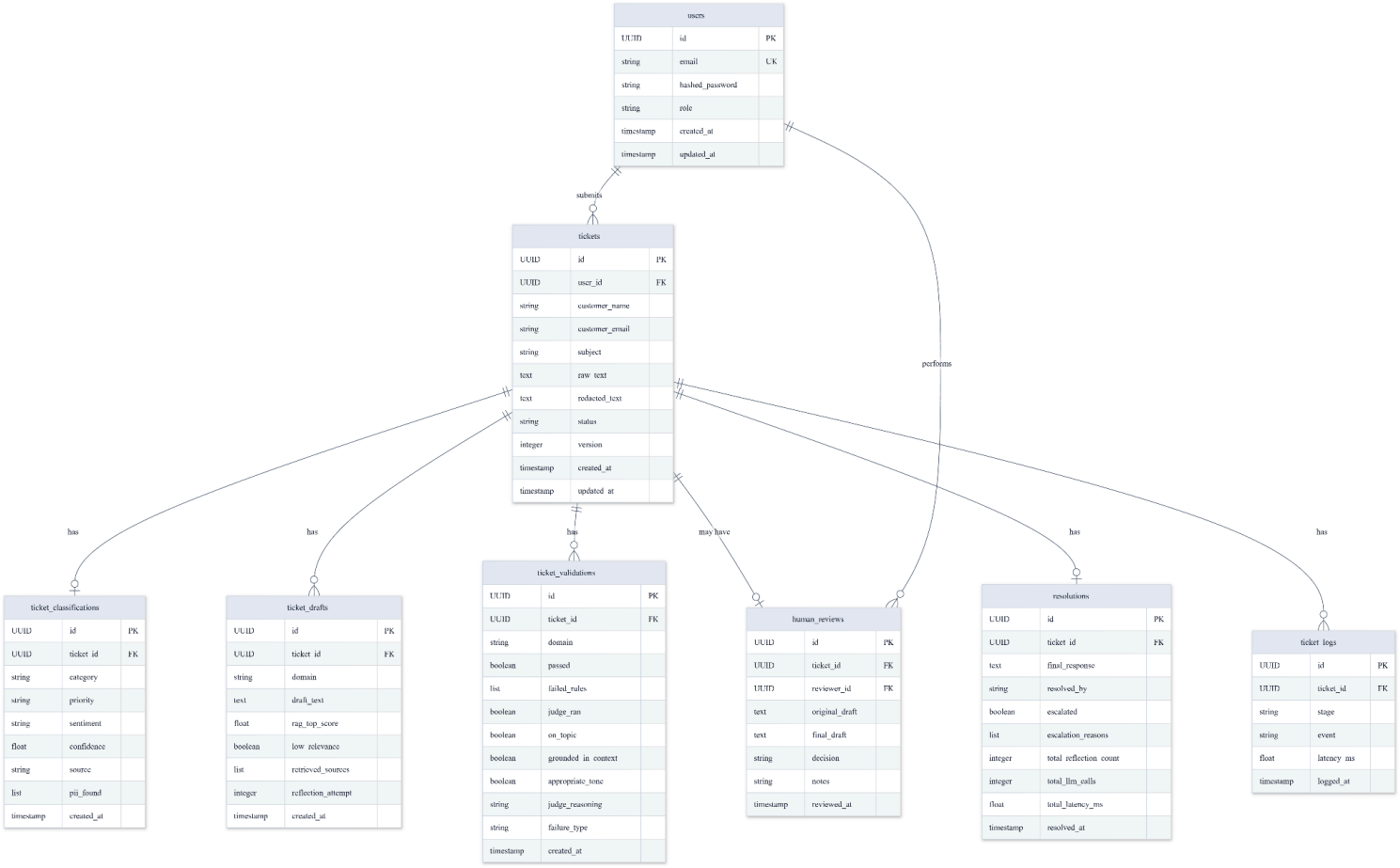


Figure 23: Entity-Relationship Diagram (ERD). The `tickets` table is the central entity. The `version` integer column enables optimistic locking. All AI pipeline artefacts (classifications, drafts, validations) are stored in child tables for a complete audit trail.

10. Size and Performance

- **Dimensioning**: The ChromaDB vector store is dimensioned to handle thousands of knowledge base documents and precedent memory vectors.
- **Latency Constraints**:
- API Gateway responses (202 Accepted) must return within 200ms.
- LangGraph background execution handles LLM API delays, with a 10-second timeout per external LLM call.
- Inference latency for the custom QLoRA adapter is constrained to operate on consumer-grade academic GPUs.

11. Quality

This section describes how the software architecture contributes to capabilities beyond core functionality: security, reliability, extensibility, and observability.

11.1 Security & PII

PII protection is a first-class architectural concern, not an afterthought. The design guarantees that raw customer PII never reaches any external API.

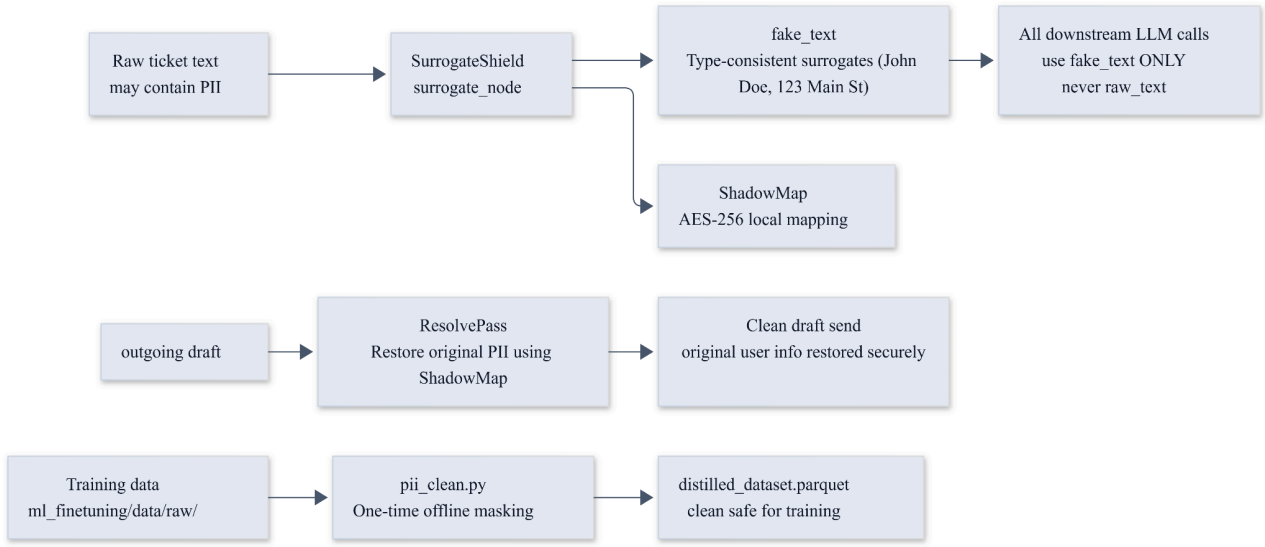


Figure 24: PII protection data flow. SurrogateShield masks PII before any LLM call. The ShadowMap restores original PII only in the outgoing response. Training data is also cleaned offline before distillation.

11.2 Observability & Logging

Component	Logging
API Gateway	Structured per-request logging; never logs PII or raw keys
ML SideCar	Per-node timing; ticket_logs table records stage + latency
ML Fine-Tuning	Inference latency per /classify call; W&B training curves
LangGraph	LangSmith traces (optional); full state logged server-side on exception
Classification Tool	WARNING logged when keyword fallback is

	used instead of Gemini
RAG Tool	WARNING logged when top score is below RAG_SCORE_THRESHOLD
Circuit Breaker	ERROR logged on open; INFO on recovery

11.3 Resilience Patterns

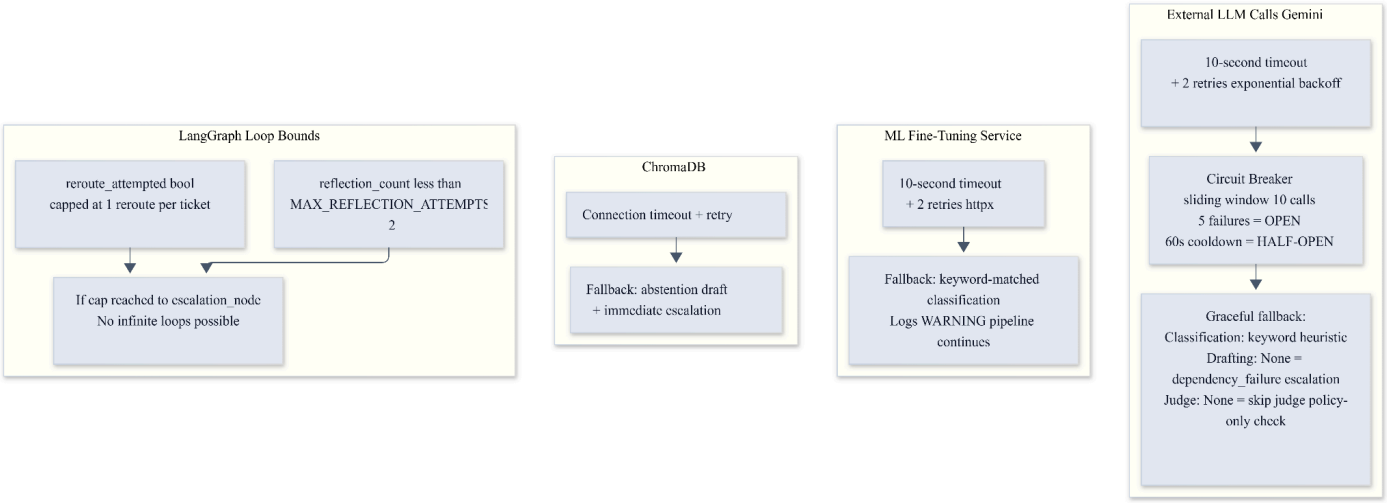


Figure 25: Resilience patterns. The circuit breaker pattern protects against cascading failures when the Gemini API is slow or unavailable. LangGraph loop bounds (reflection cap + reroute cap) guarantee every ticket terminates with a definite outcome.

Label Schema:

Field	Values
category	Technical, Billing, Account, General, Other
priority	Low, Medium, High, Urgent
sentiment	Positive, Neutral, Negative, Strongly Negative

12. RAG Pipeline - View Link

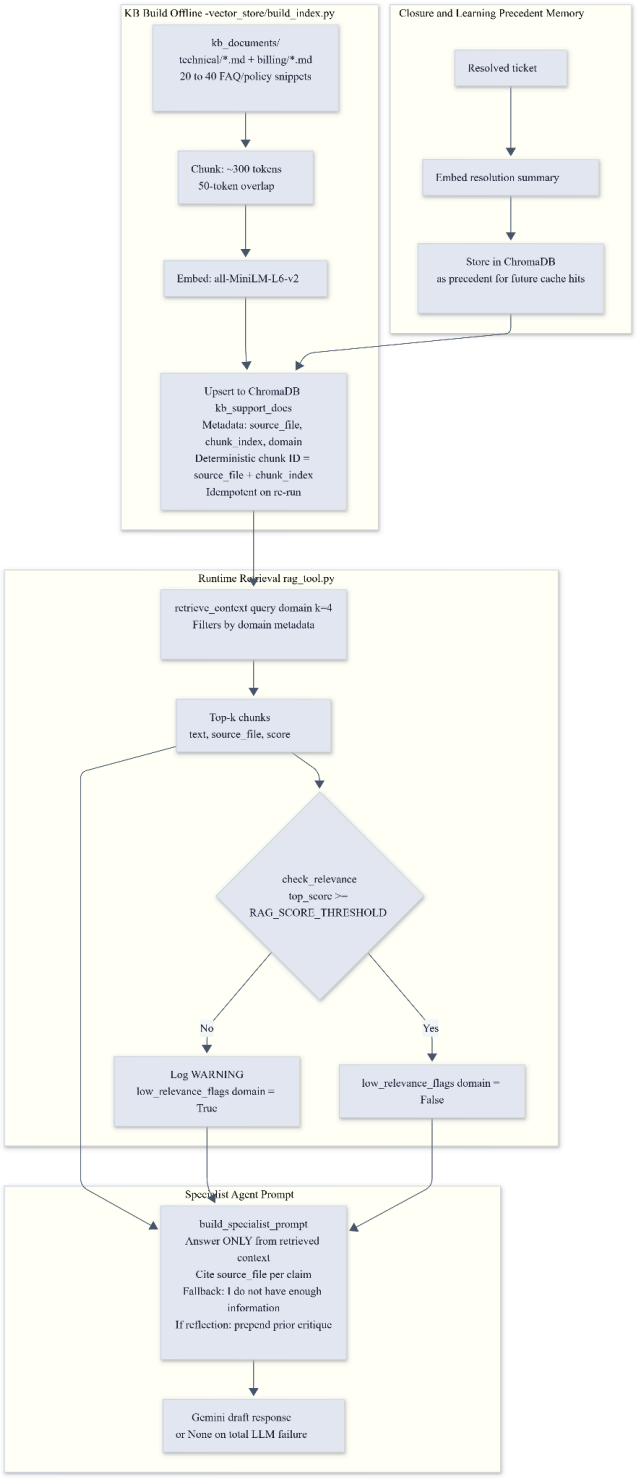


Figure 25: This diagram explains how the AI fetches knowledge to answer tickets correctly without hallucinating. It's split into three main parts:

Top Left (Offline KB Build): *This happens before the system runs. The Dev Team takes all the Markdown files (FAQs, policies), chops them into small "chunks" (about 300 tokens each), converts them into numbers (embeddings), and saves them into the ChromaDB database.*

Top Right (Closure and Learning): *Every time a ticket is successfully resolved, the system learns from it. It takes the resolution summary, embeds it, and saves it back into ChromaDB as "precedent memory" so it can answer similar tickets faster in the future.*

Middle/Bottom (Runtime Retrieval & Drafting): *When a new ticket comes in, rag_tool.py searches ChromaDB for the Top-4 most relevant chunks.*

It checks if the "relevance score" is high enough. If the score is too low, it logs a warning flag.
Finally, it gives those chunks to the Specialist Agent (Google Gemini) with strict instructions: "Answer ONLY based on this retrieved context, and cite your sources." This guarantees the AI's answer is grounded in real company policy.

13. Appendices

Appendix A: Technology Stack

Layer	Technology	Rationale
Frontend	React 18 + TypeScript + Next.js 14	Fast dev server; feature-based structure
Frontend Styling	Tailwind CSS	Rapid prototyping; clean ticket form + review screen
API Gateway	Spring Boot (Java 17/21)	JPA ORM, Optimistic locking, Enterprise Security
ML Sidecar / Orchestration	FastAPI + LangGraph (Python 3.11)	ML isolation, state graph, ChromaDB access
Auth	Spring Security JWT	Robust backend authentication
Classification LLM (production)	LLama-3.2-3B (LoRA adapter)	Open-weight, academic GPU accessible
Drafting/Judge LLM		Provider-agnostic wrapper; one config change to swap
Knowledge distillation teacher	Google Gemini 2.x Flash (JSON mode)	Cheapest bulk labeller; structured output

Fine-tuning method	QLoRA via HuggingFace PEFT +bitsandbytes	Runs on Kaggle and Colab free GPU
PII Anonymization	SurrogateShield + Python Faker	AES-256 ShadowMap; substitutes fake semantic surrogates
Vector store	ChromaDB (persistent, Docker volume)	Low operational overhead for small KB
Embeddings	sentence-transformers/all-Mini LM-L6-v2	Fast, small, no API cost
Relational DB	PostgreSQL 16	Tickets, logs, resolutions, user auth
ORM	SQLAlchemy 2.0 + Alembic	Async ORM; versioned migrations
Containerisation	Docker + Docker Compose	One-command full-stack deployment
Experiment tracking	Weights & Biases (free academic tier)	Training curves in evaluation report

Appendix B: Knowledge Distillation & RAG Pipeline - View Link

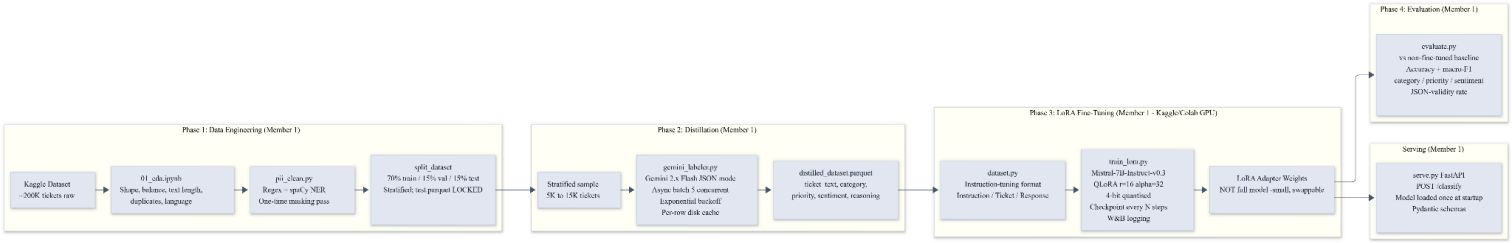


Figure 26: This diagram shows the offline process used to train Clario's classification model so it doesn't have to rely on expensive external APIs forever. It's broken into four phases:

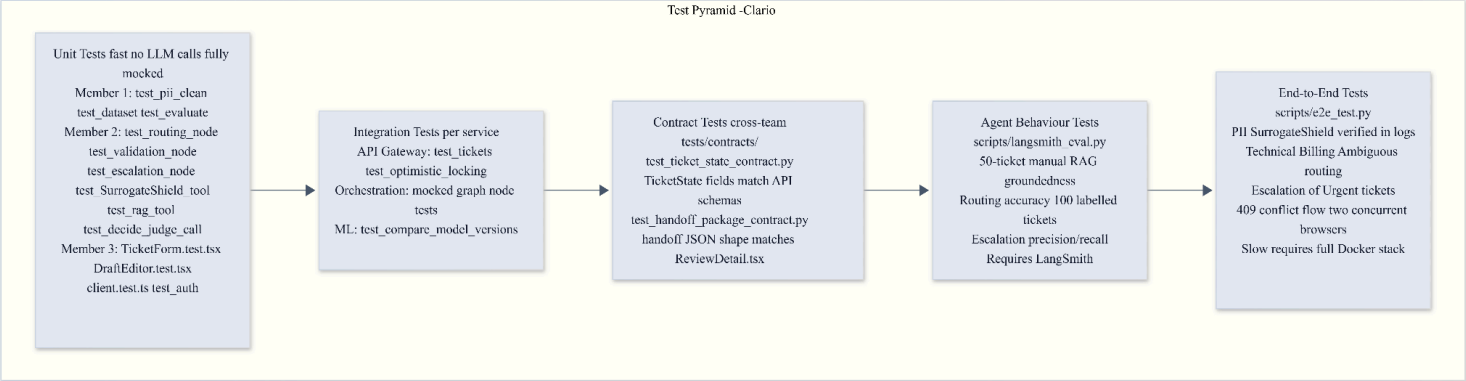
Phase 1: *Data Engineering We start with a massive dataset of ~200K raw tickets from Kaggle. We analyze them, clean them (removing all personal data using pii_clean.py), and split them into Training, Validation, and a locked Test set.*

Phase 2: *Distillation (Teacher Model) Since the raw data doesn't have the labels we need (category, priority, sentiment), we use a "Teacher" model (Google Gemini) to read a smaller sample of tickets (5K-15K) and label them for us. This creates our distilled_dataset.*

Phase 3: *QLoRA Fine-Tuning (Student Model) We take an open-source model (like Mistral-7B) and train it on a free Kaggle GPU using the labels generated by the Teacher. We use QLoRA (a memory-saving technique) so it fits on a single GPU.*

Phase 4: *Evaluation & Serving Finally, we test our fine-tuned model against the locked Test set to see if it's accurate enough. If it passes, it gets loaded into the ML Sidecar to classify tickets locally and for free!*

Appendix C: Testing Strategy



Testing Framework by Service:

Service	Framework	Key Test Files
ml_finetuning/	pytest	test_gemini_labeler, test_dataset, test_evaluate
clario-ml-sidecar/	pytest	test_surrogate_node, test_analyzer_node, test_routing_node, test_validation_node, test_escalation_node
clario-app/ (Spring Boot)	JUnit 5 + MockMvc	TicketControllerTest, OptimisticLockingTest, SecurityConfigTest
Next.js UI	Jest + React Testing Library	TicketForm.test.tsx, DraftEditor.test.tsx
Cross-service	pytest	test_ticket_state_contract, test_handoff_package_contract

Agent behaviour	LangSmith	langsmith_eval.py (routing accuracy, groundedness, escalation metrics)
Full stack	scripted	e2e_test.py (against running Docker Compose)

Appendix D: Timeline, Roles, & Evaluation

Time Line Link to View

Evaluation Metrics

Metric	Method
Fine-tuned model accuracy per task	evaluate.py on held-out test vs non-fine-tuned baseline
Macro-F1 per classification task	Same script; class-weighted to catch minority-class failures
JSON-validity rate	% of model outputs parseable as valid JSON
PII masking precision/recall	Manual spot-check of ~100 tickets vs human-labelled ground truth
RAG groundedness rate	Manual review of ~50 specialist responses; every claim traced to a chunk
Routing accuracy	Manual labelling of ~100 tickets' "correct" domain vs actual routing_decision
Automation rate	% of held-out sample resolved without escalation (from PostgreSQL logs)
Escalation precision/recall	Manual labelling of "should escalate" vs actual escalation_triggered
End-to-end latency	ticket_logs stage timestamps; P50/P95 across 50 test runs
Per-ticket LLM cost	Token counts times price from Gemini API response headers

Single-LLM ablation	scripts/ablation_single_llm.py; single prompt vs full multi-agent pipeline

*-**Ablation design:**- `ablation_single_llm.py` sends each held-out ticket as a single prompt to the same Gemini model asking it to classify, route, and draft a response in one call. Results are compared to the multi-agent pipeline on routing accuracy proxy, groundedness, latency, and cost. An honest mixed result is expected and acceptable -the proposal commits to testing whether the added agentic complexity is justified.*

Appendix E: ADRs & Stretch Goals

ADR-001: LangGraph for Orchestration

- **Decision:** Use LangGraph (not a custom loop or CrewAI) for the multi-agent orchestration.
- **Rationale:** Explicit state graph with typed state; conditional edges make the graph's branching readable and testable; bounded loop support; matches proposal commitment.
- **Consequences:** Requires Python 3.11+. `TicketState` TypedDict is the cross-team shared contract -any field change must be coordinated with all three members.

ADR-002: Explicit Domain Flip on Reroute

- **Decision:** The reroute path unconditionally flips `technical` to `billing` or vice versa rather than re-running `decide_routing()`.
- **Rationale:** Re-running the same pure function on the same inputs produces the same wrong answer -this was a confirmed bug in v2 testing on the "payment failed but money was taken" case. An explicit flip guarantees the other specialist is actually tried.
- **Consequences:** "Both" tickets can never reroute (no third domain); they escalate directly on dual-domain failure. The `reroute_attempted` boolean enforces a single-attempt cap.

ADR-003: Gated LLM Judge (Cost-Aware)

- **Decision:** The LLM-as-judge call is not run on every ticket -only when RAG score is below threshold or randomly sampled (~17.5%).
- **Rationale:** Each judge call adds ~2–5s latency and real API cost. Statistical sampling provides representative quality monitoring without running it on every ticket.
- **Consequences:** Some low-quality drafts may pass without judge review; mitigated by rule-based policy checks which always run.

ADR-004: Optimistic Locking for Human Review

- **Decision:** The `tickets` table has an integer `version` column; `PATCH /tickets/{id}/review` checks and increments it atomically; returns HTTP 409 on version mismatch.
- **Rationale:** Multiple human agents can open the same escalated ticket simultaneously. Silent last-write-wins overwrites are a real support team data-loss bug.

- **Consequences:** Frontend must handle 409 by preserving `draftBuffer` and re-fetching rather than discarding in-progress edits. `ConflictBanner` component required.

ADR-005: LoRA Adapters Not Merged Weights

- **Decision:** Save and deploy LoRA adapter weights separately from the base model, not merged.
- **Rationale:** Adapter files are ~50–200 MB vs 13–30 GB for a merged 7–8B model; faster iteration; easier to swap; no need to re-download the base model per experiment.
- **Consequences:** `serve.py` must load both base model and adapter at startup. Startup time is longer but per-request latency is identical. Adapters stored on HuggingFace Hub private repo (not in Git -too large).

ADR-006: Semantic Caching -Deferred

- **Decision:** `cache_check_node` is structurally in the graph but intentionally minimal at v1. Full semantic caching is deferred.
- **Rationale:** Getting the core pipeline correct (reroute fix, validation, escalation) is higher priority. The cache node's position in the graph does not change when the logic is filled in.
- **Consequences:** Near-duplicate tickets will go through the full pipeline until this is implemented.

ADR-007: JWT in localStorage (Acknowledged Risk)

- **Decision:** Frontend stores JWT in `localStorage` for the academic demo.
- **Rationale:** `httpOnly` cookie auth requires backend session support not in scope for this project.
- **Mitigation:** Explicitly documented in `frontend/README.md` and the evaluation report as a known XSS risk, acceptable for an academic symposium demo, not recommended for production. Examiners who notice this should see it acknowledged proactively.

14. Stretch Goal

- To process multimodal inputs by integrating advanced voice-to-text transformations(English Language) and image classification models.
- Rysera STEM LMS Integration

15. References

- T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," *arXiv preprint* arXiv:2305.14314v1, May 2023.
- LangChain AI, "LangGraph: Multi-Agent State Workflows," 2024. [Online]. Available: https://langchain-ai.github.io/langgraph/
- P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- S. V. Jathanna, "SurrogateShield: Beyond Redaction for High-Utility, Privacy-Preserving LLM Interactions," *arXiv preprint* arXiv:2606.29567v1, Jun. 2026.
- L. Zheng et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2023. [Online]. Available: arXiv:2306.05685.

- J. Shen, "Evidence Graph Consistency in RAG: A Model-Dependent Analysis of Hallucination Detection," *arXiv preprint* arXiv:2606.06748v2, Jun. 2026.
- X. Xu et al., "A Survey on Knowledge Distillation of Large Language Models," *arXiv preprint* arXiv:2402.13116v4, Oct. 2024.
- E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," *arXiv preprint* arXiv:2106.08685, Jun. 2021.
- Y. Li et al., "Mitigating Hallucination in Large Language Models (LLMs): An Application-Oriented Survey on RAG, Reasoning, and Agentic Systems," *arXiv preprint* arXiv:2510.24476v1, Oct. 2025.
- S. Han and W. Choi, "Multi-Agent Clinical Decision Support System for KTAS-Based Triage," *arXiv preprint* arXiv:2408.07531v2, Aug. 2024.
- Z. Yu et al., "Triangle: Empowering Incident Triage with Multi-LLM-Agents," Microsoft Research, *arXiv preprint*, Feb. 2025.